PER%SENSE — LEGACY DOS SOURCE

Amortization Engine Code Walkthrough

A maintainer's guide to the loan amortization code flow, the field-presence solver model, and the underlying financial mathematics.

Scope. The amortization calculation engine (`AMORTOP.pas`), its controller/dispatcher (`Amortize.pas`), the supporting date & interest mathematics (`INTSUTIL.pas`, with primitives in `VIDEODAT.pas` / `PETYPES.PAS`), and the screen layer that feeds the engine (`AmortizationScreenUnit.pas`).

Audience. Engineers maintaining or porting the financial logic. Every claim is anchored to a `file:line` reference for independent verification.

Authority. The DOS source is the authoritative implementation of the financial logic. Line numbers cite the documented tree `legacy/src_documented/dos_source/` — the same routines as the canonical tree, annotated with explanatory comment blocks above each procedure.

Location. `legacy/code_docs/amortization_dos_walkthrough.pdf`

Contents

0 How to read this document

Per%Sense is a financial calculator whose defining feature is **field-presence dispatch**: the user fills in the fields they know, leaves blank the one they want, and the program selects a formula path to solve for whatever is missing. The amortization screen applies this to loans — leave the payment blank to size a payment, leave the rate blank to back out a rate, leave the term blank to find a payoff date, and so on. Understanding the code means understanding (a) how a blank field becomes a solve request, (b) how the engine walks a repayment schedule forward, and (c) how it inverts that walk to solve backward.

Citation convention

Inline references look like `AMORTOP.pas:723`. Unless a range is given, the number is the line where the construct begins. All line numbers refer to the documented tree `legacy/src_documented/dos_source/`.

> **Documented vs canonical tree.** The documented tree carries the same routines as the canonical `legacy/src/dos_source/`, with explanatory comment blocks inserted above each procedure. Those comments shift the line numbers (e.g. `AMORTOP.pas` is 1942 lines here vs 1656 canonical), so cite and verify against `src_documented/` when reading this document.

> **If you read only one section, read §6.2 (ComputeNext).** Every forward schedule and — through Iterate — every backward solve ultimately runs that one per-period routine. The rest of the engine is setup, dispatch, and inversion around it.

1 Architecture & module map

1.1 The four layers

The amortization feature is a clean four-layer stack. Data flows down on input and back up on output; the global model records in the middle are the shared currency.

```
+-------------------------------------------------------------------+
| UI LAYER       AmortizationScreenUnit.pas                         |
|                grids of cells --> Assign*Values --> records       |
+--------------------------------+----------------------------------+
                                 |  global model (PEDATA.pas):
                                 |  h, w, pre[], balloon[], adj[], mor, targ, skp, fancy, nlines
+--------------------------------v----------------------------------+
| CONTROLLER     Amortize.pas                                       |
|                FirstPass -> SufficientDataOnScreen -> Enter       |
|                -> MakeTable (emit schedule)                       |
+--------------------------------+----------------------------------+
                                 |  solves the one blank field
+--------------------------------v----------------------------------+
| ENGINE         AMORTOP.pas                                        |
|                RepayFancyLoan / RepayLoan (forward walk)          |
|                ComputeNext (one period)    Iterate (backward solve) |
+--------------------------------+----------------------------------+
                                 |
+--------------------------------v----------------------------------+
| MATH           INTSUTIL.pas (+ VIDEODAT/PETYPES)                  |
|                Round2 . YearsDif . AddPeriod . exxp/lnn . Rate-Yield |
+-------------------------------------------------------------------+
```

Two routines are the seams worth memorizing. `Amortize.pas:1104` `Enter(code)` is where the controller decides what to solve. `AMORTOP.pas:723` `Paymenttype.ComputeNext` is where one period of a loan is actually computed. The controller never touches a period directly; the engine never decides what the unknown is. That separation is the whole architecture.

1.2 The field-presence philosophy

Every input field carries both a value and a status. The status is an `inout` code, ordered by "hardness" `PETYPES.PAS:178,216–218`:

Status	Value	Meaning
`empty`	0	Blank cell — the user left it out. This is the solve request.
`outp`	1	Computed output — the engine filled it.
`defp`	2	Defaulted / known — supplied by a default rule.
`inp`	3	Hard user input — the user typed it.

The controller reads which field is empty and that selects the formula path. "Known" throughout the code means `status >= defp`. The hardness ordering also governs penny-rounding: only a genuinely hard payment (`inp`) triggers rounding of interest to the cent during the walk (§9).

1.3 End-to-end data flow

```
user edits / blanks a cell
      |
      v
[Verify*CellString] reject out-of-range (rate >= 100, points >= 10, negative counts)
      | ok
      v
CellAfterEdit -> Assign*Values(record, col, text, inp)
      |    blank text => field := 0 / unkdate , status := empty      (the solve request)
      |    typed text => field := parse(text) , status := inp
      v
PEDATA globals: h . w . pre[] . balloon[] . adj[] . mor . targ . skp   (+ fancy, nlines)
      |
   Enter key / Calculate
      v
DoCalculation: publish nlines (used-row counts) ; Enter(no_tab)
      v
Amortize.Enter -> FirstPass (classify) -> dispatch (solve the blank field)
      |                                          +-> Iterate -> RepayFancyLoan -> ComputeNext
      |    solved field gets status := outp
      v
AllAMZData2Grids -> *Values2Grid    (output cells coloured as computed)
      +-> TableOutput -> MakeTable -> schedule rows (window / .csv / .txt)
```

2 The data model

2.1 Status codes & sentinels

Beyond the four `inout` status codes (§1.2), the model leans on a set of "magic" numeric sentinels that stand in for non-numbers, and a small set of tolerances. These are defined in `PETYPES.PAS`:

Sentinel	Value	Meaning	Source
`error` / `unk`	−8888	Parse / range error; `unk` ("?") shares the value	`PETYPES.PAS: 191,192`
`blank`	−7777	Empty / field left blank	`PETYPES.PAS:193`
`unkbyte` / `errorbyte`	−88 / −99	Unknown / error in a date's month field	`PETYPES.PAS: 203,223`
`unkdate`	(d:0; m:−88; y:0)	The unknown date	`PETYPES.PAS:204`
`latest`	(d:1; m:12; y: 249)	Blank / "forever" date sentinel	`VIDEODAT.pas:68`

The guard `ok(v)` `INTSUTIL.pas:770` returns true iff `v` is none of `unk`/`blank`/`error` — the standard "is this a real number?" test. `dateok(f)` `INTSUTIL.pas:758` is true iff 1 ≤ `f.m` ≤ 12.

Tolerance	Value	Primary use
`teeny`	1×10^{-10}	The half-cent deficit in Round2 (§5.1); the near-zero rate / convergence cutoff. `PETYPES.PAS:225`
`small`	1×10^{-4}	The Taylor-series switch threshold in exxp/lnn (§5.5); the "a penny" scale. `PETYPES.PAS:224`
`half`	0.5	Series coefficient; also used to avoid integer overflow in the Rule-of-78 base. `PETYPES.PAS:226`

2.2 The loan record & advanced options

The screen's model lives in a handful of global pointers declared in `PEDATA.pas` `PEDATA.pas:257–277`:

Global	Type	Holds
`h`	AMZptr → AMZLoan	The core loan: amount, loan date, rate, first-payment date, #periods / last date, payments-per-year, payment amount, points, APR — each with a paired status field.
`w`	^balloonrec	The payoff / "balance as of" query (a date + amount pair).
`pre[]`	prepaymentarray	Extra periodic payments: startdate, nn (count), stopdate, peryr (freq), payment, and a rolling nextdate cursor.
`balloon[]`	balloonarray	One-time lump payments: date + amount (each with status).
`adj[]`	adjarray	ARM rate/payment changes: date, loanrate, amount (new payment), plus an amtok flag.
`mor`	moratoriumptr	Interest-only deferment until a first_repay date.
`targ`	targetptr	Minimum principal-reduction target per payment.
`skp`	skipptr	Skipped-payment months as a raw string (e.g. `"6-8,12"`).
`fancy`	boolean	Global flag `PEDATA.pas:123`. When set, the engine consults all advanced-option records and runs the period-by-period walk.
`nlines`	blockarray	Per-block used-row counts `PEDATA.pas:233`, published before each calc so the engine knows how many advanced rows are live.

> **1-based records, 0-based grids.** The pre/balloon/adj arrays are 1-based (`for i := 1 to max...`) but the on-screen grids are 0-based. The mapping `array[ARow+1]` recurs in every advanced-option marshaller. Off-by-one here is the classic porting trap.

2.3 The paymenttype engine object

Inside the engine, one amortization row in progress is a `paymenttype` object `AMORTOP.pas:87–90`. Its fields carry the running state of the walk:

Field	Meaning (units: dollars / dates)
`base_date`	The last regular payment date. Drives AddPeriod to the next regular date. Advanced only when a regular payment actually occurs.
`date`	The current payment date (may be an off-grid balloon date).
`prevdate`	The last payment date of any kind — drives the interest-accrual interval.
`payamt`	Total cash paid this period (regular ± balloon / prepay / target).
`interest`	Interest accrued since prevdate.
`principal`	Remaining principal balance after this payment.
`usaprinc`	The USA-rule unpaid-interest bucket after this payment (§6.4).
`paynum`	1-based payment counter.

Two instances are kept: `payment` (the previous step, saved) and `nextpayment` (the one being advanced). Only `nextpayment.ComputeNext` moves the walk forward; `payment` is the rollback anchor. A companion object `saved_balloon_state` `AMORTOP.pas:68` deep-copies the balloon/prepay/adjustment cursors so a speculative walk inside `Iterate` can be undone — including heap copies of every active prepayment series, because `ComputeNext` mutates them in place.

3 The UI layer: cell → record + status

The screen is `TAmortizationScreen` `AmortizationScreenUnit.pas:118`, a VCL MDI child hosting a grid of `TPersenseGrid` controls. Its single job is to translate between on-screen cells and the global model records of §2.2 — converting a blank cell into a `status := empty` solve request and a typed cell into a hard value.

3.1 Grids & columns

The main loan terms live in `AmortGrid` (one row). The advanced options each get their own grid, shown only when `fancy` is on; the payoff query grid is always visible. Column indices are named constants `AmortizationScreenUnit.pas:86–`:

Grid	Columns (index)
`AmortGrid`	Amount(0) LoanDate(1) LoanRate(2) FirstDate(3) NPeriods(4) LastDate(5) PerYr(6) PayAmt(7) Points(8) APR(9, read-only)
`PayoffGrid`	Date(0) Amount(1)
`AdjustmentGrid`	Date(0) Rate(1) Amount(2)
`BalloonGrid`	Date(0) Amount(1)
`PrepaymentGrid`	Start(0) NN(1) Stop(2) PerYr(3) Payment(4)
Moratorium / Target / Skip	single cell (col 0)

3.2 The blank-vs-typed mapping

`AssignAMZLoanValues` `AmortizationScreenUnit.pas:1573` is the canonical cell→field+status mapper, a `case ACol of` with one block per column. Every block follows the same blank-vs-typed shape — here is the Amount column `:1578`:

```
AMZAmountCol: begin
  if( Value = '' ) then begin
    AMZ.amount := 0;
    AMZ.amountstatus := empty;            { BLANK cell -> the solve request }
    AmortGrid.SetCellHardness( ACol, 0, empty );
  end else begin
    AMZ.amount := StringFormat2Double( Value, IsError );
    AMZ.amountstatus := Hardness;         { typed -> inp (or pasted hardness) }
    AmortGrid.SetCellHardness( ACol, 0, Hardness );
  end;
end;
```

Per-column parsing differs by type: dollars via `StringFormat2Double`; dates via `StringFormat2Date`; the loan rate, points and APR are parsed and divided by 100 (percent → fraction); #periods and payments-per-year via `StringFormat2Int`. The inverse routine `AMZValues2Grid` `:1696` multiplies the rate/points/APR back by 100 for display and colours each cell by its status, so a solved (`outp`) cell renders visibly as computed.

3.3 Advanced options input

Each advanced grid has its own `Assign*Values` routine following the same blank-vs-typed pattern, dereferencing the 1-based array at `array[ARow+1]`:

- **Prepayment** `:2020`: start/stop dates, count, frequency, and a payment parsed with `StringFormat2Int` at `:2078` — whole dollars, unlike the loan's PayAmt which is `StringFormat2Double`. A real porting gotcha.
- **Balloon** `:1942`: date + amount.
- **Adjustment (ARM)** `:1847`: date, rate (÷100), new payment.
- **Moratorium** `:2190`: a single first_repay date.
- **Target** `:2146`: a single minimum-principal-reduction amount.
- **Skip-months** `:2233`: the raw string is stored verbatim (`Skip.skipmonths := Value`); it is parsed into month numbers later, in the controller.

> **Fancy is an explicit toggle in the DOS source.** Supplying an advanced value does not auto-enable fancy mode here. The advanced grids are only editable when the user has toggled Advanced on (`AdvancedIsOn` returns `fancy`, `:487`), and the engine gates every advanced behaviour on `if (fancy)`. The Go port deliberately inverts this — it sets `input.Fancy = true` whenever any advanced option is present. Keep the difference in mind when comparing outputs.

3.4 Validation & the calc trigger

`Verify*CellString` routines reject bad input before it commits, reading the raw percent value (pre-÷100): loan rate in (−100, 100), points < 10, counts ≥ 0; adjustment rate in (−100, 100). `AmortGridCellBeforeEdit` `:1529` enforces the #periods ↔ last-date mutual exclusion: typing a last date clears #periods and vice-versa, so the dispatch never sees both ways of expressing loan length at once.

Pressing Enter, or the Calculate command, routes to `DoCalculation` `AmortizationScreenUnit.pas:615`:

```
nlines[AMZAdjBlock]     := AdjustmentGrid.GetUsedRowCount();
nlines[AMZballoonblock] := BalloonGrid.GetUsedRowCount();
nlines[AMZpreblock]     := PrepaymentGrid.GetUsedRowCount();
nlines[AMZBalanceBlock] := PayoffGrid.GetUsedRowCount();
Enter( no_tab );          { <-- into the controller, Amortize.pas }
AllAMZData2Grids();       { <-- results back to the grids }
```

It publishes the live row counts into `nlines` `:619–622`, calls the controller's `Enter(no_tab)` `:623`, then pushes the (now partly `outp`) records back to the grids. The printed schedule takes a parallel path through `TableOutput` → `MakeTable`.

4 The controller: classify, gate, dispatch

The controller turns "which field is blank" into "which solver to run." Three routines do the work in order: FirstPass classifies, SufficientDataOnScreen gates, and Enter dispatches.

4.1 FirstPass — classification

`FirstPass` `Amortize.pas:271` walks the inputs and sets the flags the dispatch reads. Key outcomes:

- Default first-payment date `:294`: if blank but loan date + pmts/yr are present, derive it (`DefaultFirstPaymentDate`, one full period after settlement).
- The term pair `:300–307`: if first-date + #periods are present, compute lastdate via `AddNPeriods`; if first-date + last-date are present, compute nperiods via `NumberOfInstallments` (a negative count is the "last before first" error); if neither, set `lastok := false`. `lastok` is the central "is the term known?" flag.
- Advanced prep: parse skip-months into `skipmonthset` via `MonthSetFromString` `:204`; `CheckPrepayments` classifies the prepay rows and sets `unkpre`; snap adjustment dates onto payment dates; `SortAdj`/`SortBalloons`.
- The hard-payment flag `:409`: `hard_payment := (h^.payamtstatus = inp)` — the switch that later enables penny-rounding of interest (§9).

4.2 SufficientDataOnScreen — the gate

`SufficientDataOnScreen` `Amortize.pas:1049` returns true only if a solve is actually possible: payments-per-year present, and (rate or payment) present, and both dates present, and (amount present or computable) with adjustments resolvable. When the term is unknown (not `lastok`), it verifies there is enough information to derive it before letting Enter proceed.

4.3 Enter — control flow

`Enter(code: byte)` `Amortize.pas:1104` (body at `:1428`) orchestrates everything. The notable beats:

1. Fast path: if nothing is dirty since the last calc and a table was requested, jump straight to printing — no recompute.
2. FirstPass, then a with_tab early-out if the user merely tabbed and nothing changed.
3. Sufficiency gate, then a second FirstPass (clearing output cells destroyed the last-payment date).
4. Validation & geometry: two-payment minimum; date ordering; balloon / moratorium placement; `f := GrowthPerPeriod`; prepaid auto-detection; moratorium setup (`repay_from`, `prorate`, `nrepay`); in-advance + adjustments rejection; `DetermineVeryLast`.

5. The dispatch — §4.4.
6. Finishing: optional tacked-on final balloon; per-adjustment payment/rate solves; APR-with-points; balance-as-of; the "Dav Holle provision" that hardens balloon/adjusted-payment amounts to the cent when the top-line payment is hard `:1687`.

> **Enter does not print.** It returns after solving (`if (code <> make_table) then exit`). Schedule emission is split into `MakeTable` `Amortize.pas:1718`, which calls `Enter(no_tab)` first and then walks the schedule (§6.6).

4.4 The dispatch table

The heart of the controller is a priority cascade keyed first on whether the payment is known (`h^.payamtstatus >= defp`, `Amortize.pas:1577`), then on the remaining unknown:

Blank field (the unknown)	Guard	Solver	Line
Loan rate	payment known & rate blank	`EstimateAndRefineRate`	`1589`
Balloon amount	`unkballoon > 0`	`EstimateAndRefineBalloon`	`1593`
Term (last date / #periods)	not `lastok`	`DetermineLastPaymentDate` (AMORTOP: 1589)	`1602`
Prepayment amount	`unkpre>0` & payment blank	`EstimateAndRefinePeriodicPrepayment`	`1605`
Prepayment duration	`unkpre>0` & stop-date blank	`DeterminePrepaymentDuration`	`1614`
Amount borrowed	`ComputeLoanAmount`	`EstimateAndRefineLoanAmount`	`1618`
Payment amount	payment blank (else-branch)	`EstimateAndRefinePayment`	`1632`
Per-adjustment payment / rate	post-dispatch loop	`EstimateAndRefineAdjPayment` / `AdjRate`	`1665,1671`
APR (points entered)	`pointsstatus > defp`	`EstimateAndRefineAPRwithPoints`	`1675`

The order is a priority list, not a mutually-exclusive classification: rate beats balloon beats term beats prepayment beats amount. The payment-known vs payment-blank fork is the top-level split. Each solver writes its result's status as `outp` so the UI later renders it as computed.

5 The mathematics primitives

Everything the engine computes rests on a small set of primitives in `INTSUTIL.pas`, with date serialization in `VIDEODAT.pas`. These are deliberately hand-rolled to control rounding and precision — the financial behaviour is defined by these implementations, not by the standard library.

5.1 Round2 — round-half-DOWN

All monetary rounding goes through `Round2` `INTSUTIL.pas:346`:

```
procedure Round2(var x :real);
          const halfpenny : real = 0.005 - teeny;          { teeny = 1e-10 }
          begin
          if (x>0) then x := x + halfpenny
          else x := x - halfpenny;
          x := Trunc(x*100)/100;
          end;
```

Round2(x) = Trunc(100 · (x + sgn(x)·(0.005 − teeny))) / 100. A nudge of just under half a cent is added away from zero, then the value is truncated toward zero at the second decimal. The −teeny deficit is the whole point: it ensures an exact half-cent is not promoted to the next cent.

> **Worked example.** Round2(1.125). With a true half-penny: 1.125 + 0.005 = 1.130 → Trunc(113.0) = 113 → 1.13 (half rounds up). With Per%Sense's halfpenny = 0.0049999999: 1.125 + 0.0049999999 = 1.1299999999 → Trunc(112.99...) = 112 → 1.12. The exact half-cent truncates downward. This is the documented round-half-down rule — not banker's rounding and not commercial half-up. It is symmetric about zero, so −1.125 → −1.12.

5.2 Dates & the Julian epoch

A `daterec` is (d, m, y) where y is a byte offset, not a calendar year. The valid range is y ∈ [0,249]; the Go port documents the epoch as 1900 (calendar year = 1900 + y). The sentinel y = 249 is the blank / "forever" date. `Julian` `VIDEODAT.pas:470` maps a date to a serial day number:

Julian = ⌊ (1461·y − 1) / 4 ⌋ + daysbefore[m] + d, where 1461 = 4·365 + 1 is the days in a four-year leap cycle, `daysbefore[m]` is the cumulative day count before month m (swapped between leap and non-leap tables), and the `div 4` is integer division — this integer-division behaviour must be reproduced exactly. `MDY` `VIDEODAT.pas:490` is the inverse.

`ExtendedJulian` `INTSUTIL.pas:938` is the synthetic 30/360 serial used when counting whole months: **ExtJulian360 = 360·y + 30·m + d**. For any non-360 basis it falls back to true Julian. The basis enum is `basistype = (x365, x360, x365_360)` `PETYPES.PAS:514`.

5.3 YearsDif — the day-count conventions

`YearsDif(z, a)` `INTSUTIL.pas:996` returns the signed number of years from a to z. It is the most consequential primitive: a one-day difference in the convention compounds across a whole schedule. There are several branches plus a sign-recursion (`if a > z then YearsDif := −YearsDif(a,z)`) so the asymmetric corrections always apply consistently.

Branch A — 30/360 basis (`basis=x360`): `til = (zy − ay) + (zm − am)/12 + (zd − ad)/360` with three corrections from "Standard Securities Calculation Methods" p.16: if start day = 31 and end < 31, add 1/360; if start day = 30 and end = 31, subtract 1/360; and the February special case — if start month = Feb and start day > 27, subtract (30 − ad)/360.

Branch B — PVL/CHR screens or the x365_360 basis: `YearsDif = (Julian(z) − Julian(a)) · yrinv` with `yrinv = 1/yrdays`. Actual Julian days over the year length set by `SetYrDays` — 365.25 for these screens. For the x365_360 basis the numerator is actual days but yrdays = 360: the "365/360" hybrid.

Branch C — loan mode (iAMZ): the 365/366 actual rule. Counts actual Julian days but picks the per-year denominator from the start year of each segment: a leap start year (`a.y mod 4 = 0`) uses 366, else 365. When the span crosses calendar years it decomposes into whole interior years + the fraction of the start year + the

boundary day + the fraction of the end year (by recursion). Each fractional piece divides actual days by that segment's own 365 or 366.

> **This is the amortization default.** The loan screen runs in `iAMZ` mode, so unless the user selects a 30/360 or 365/360 basis, schedule interest uses this actual/actual-with-start-year-denominator rule. Any port that reaches for a single fixed day-count will drift from the DOS output.

5.4 Advancing dates

`AddPeriod(f, peryr, orig_day, negative)` `INTSUTIL.pas:1543` steps a date by one payment period. Weekly/biweekly (26, 52) use pure Julian day arithmetic (364 div peryr days). Semimonthly (24) jumps 15 days with month-end snapping. Monthly through annual restore the canonical day (`d := orig_day`) then shift the month by `12 div peryr`. Every branch finishes with `CheckForDaysTooLarge`, which clamps the day to the month length — so Jan 31 + 1 month → Feb 28/29, and orig_day lets the schedule recover Mar 31 at the next long month.

`AddDays` `:1157` is a plain Julian round-trip. `AddNPeriods` `:1763` advances n whole periods. `AddYears` `:1129` caps at ±128 years and, off the 360 basis, advances round(yrs · yrdays) actual days. `NumberOfInstallments(f, l, peryr, z)` `:1210` counts whole payments between two dates and, as a side effect, snaps `l` onto an exact payment date; the relation code `z` controls the snap direction. The count finishes with an inclusive `succ(...)` — converting a count of intervals into a count of payments (both endpoints pay).

5.5 exxp / lnn / Power

Continuous compounding needs exp and ln, but the Turbo Pascal library versions were neither guarded nor precise near the origin, so the code rolls its own.

`exxp(x)` `INTSUTIL.pas:1460`: returns 0 (and raises overflowflag) for x > 70; returns 1e−32 for x < −70; for |x| < small uses the cubic Maclaurin series; else the library exp. $\mathbf{e^x \approx 1 + x + \tfrac{1}{2}x^2 + \tfrac{1}{6}x^3}$ ($|x| < 10^{-4}$).

`lnn(x)` `:1487`: errors for x ≤ 0; for |x−1| < small uses a series in t = x−1; else the library ln. $\mathbf{\ln(x) \approx t - \tfrac{1}{2}t^2 + \tfrac{1}{3}t^3}$, t = x−1.

> **Why small = 1e−4.** The source comment is explicit: "Turbo compiler bug — ln goes to 0 too fast for x close to 1! The effect begins to be noticeable at about 1.0001." For near-zero periodic rates (where 1 + i ≈ 1) the native ln lost precision; the series restores it.

`Power(x, n)` `:331` is built on these: `Power := exxp(n · lnn(x))` for x > 0, else 0 — inheriting the overflow and precision guards.

5.6 Rate conversions

The internal rate is a continuously-compounded force of interest; the displayed "yield" is the equivalent periodically-compounded rate. `RealPerYr(n)` `INTSUTIL.pas:1597` gives periods-per-year for a frequency code (daily uses yrdays; weekly/biweekly derive from it; the Canadian flag masks off bit 128).

YieldFromRate(rr, n) = nn · ($e^{rr/nn}$ − 1), nn = RealPerYr(n) `:1612`. **RateFromYield(yy, n) = nn · ln(1 + yy/nn)** `:1624`. These are exact inverses. `ReportedRate` / `InterpretedRate` `:1878,1890` re-express a rate between the header frequency and the current screen frequency, but only when Canadian or daily compounding is active. `SetYrDays` `:480` sets yrdays = 365.25 for the x365 basis and 360 otherwise.

6 Forward schedule generation

A forward schedule answers: given all loan terms, what is the period-by-period interest/principal/balance? Everything else — printing, and (through Iterate) every backward solve — is built on this walk.

6.1 The growth factor

`GrowthPerPeriod` `AMORTOP.pas:1481` returns the per-period balance multiplier f = 1 + (rate per period):

```
weekly (52):   f = 1 + 7  * yrinv * rate
biweekly (26): f = 1 + 14 * yrinv * rate
else:          f = 1 + rate / RealPerYr(peryr)
```

So `f - 1` is the periodic interest rate — the workhorse multiplier the rest of the engine calls `f_1`. Weekly/biweekly use an explicit 7- or 14-day year fraction so their day count matches a 365 basis; all other frequencies use the nominal rate / periods-per-year.

6.2 ComputeNext, step by step

`Paymenttype.ComputeNext(var p, usap)` `AMORTOP.pas:723–822` advances `nextpayment` by exactly one period. This is the single most important routine in the engine; the Go port mirrors its order of operations exactly. The six steps, in order:

Step 1 — advance the date `:755`: `date := base_date; AddPeriod(date, peryr, firstdate.d, add)` — step the regular grid one period from the last regular date, anchored to the first-payment day-of-month.

Step 2 — skip-month zeroing `:756`: `if (date.m in skipmonthset) then payamt := 0 else payamt := d`.

Step 3 — find the next extra / balloon `:757–782`: `FindNextExtra` `(:629)` returns the soonest balloon and/or coinciding prepayment (co-dated series are summed), with a bitmask `xsource` of contributors. `balloonpos := DateComp(nextextra.date, date)` positions it: −1 before (pay only the extra; do not advance the regular grid), 0 coincides (add or replace per `plus_regular`), 1 after. `CheckOffBalloon` `(:694)` consumes the used extra and advances its cursor. The regular grid (`base_date`) advances only when a regular payment occurred (`balloonpos >= 0`).

Step 4 — the interest interval, then accrual `:786–798`: for the 360 basis or non-exact mode the interval snaps to whole calendar months (with a half-month nudge for semimonthly); otherwise it is the exact day-count `YearsDif(date, prevdate)`. Interest then accrues on the principal net of the USA bucket (`p - usap`):

```
non-daily: interest = rate * t * (p - usap)              (simple)
daily:     interest = (e^(truerate*t) - 1) * (p - usap)  (continuous)
```

where t is the interval in years. Then `if (hard_payment) then Round2(interest)` — penny-rounding is applied only when the payment is hard, so it is suppressed during iteration (§7.2, §9).

Step 5 — moratorium & target `:800–816`: during a moratorium (date before `first_repay`) only interest is collected — principal is untouched. Otherwise, if this period's principal reduction (`payamt - interest`) falls short of the minimum `targ^.target`, the payment is bumped up to meet it.

Step 6 — new balance & the USA rule `:821`: `p := p + interest - payamt`. Under the USA rule a parallel bucket tracks unpaid interest: `usap := usap + interest - payamt`, floored at 0. Principal whose accrued interest has not yet been covered (the bucket) does not itself earn interest — which is exactly why step 4 accrues on `p - usap`.

> **Mental model.** One ComputeNext call = advance the clock one period → decide what is paid (regular, skip, balloon, target, moratorium) → accrue interest over the elapsed time → subtract the payment from the balance. Repeat until the balance hits zero. Every schedule, forward or backward, is this loop.

6.3 Plain vs fancy

Two forward drivers exist. `RepayLoan` `AMORTOP.pas:1520` is a fast closed-form balance recurrence for plain (non-fancy, 360-basis-or-non-exact) loans — no rows are built, it just iterates the balance over `nperiods`. `RepayFancyLoan` `AMORTOP.pas:1331` is the general period-by-period walk: it drives `NextPayment.ComputeNext` repeatedly, consults every advanced option, optionally accumulates discounted cashflows for the APR solver, optionally prints, and applies ARM adjustments via `Re_Amortize` when a payment date crosses an adjustment date. It is invoked silently (Output = nil) from inside `Iterate` and with an output list from `MakeTable`.

6.4 Interest models in one place

Model	Per-period interest	Where
Simple / compound (arrears)	`rate · t · (p − usap)`	`AMORTOP:796`
Continuous (daily)	$(e^{truerate \cdot t} - 1) \cdot (p - usap)$	`AMORTOP:794`
Interest-in-advance (annuity-due)	`(p − d) · (f−1) / (2 − f)`	`AMORTOP:1528`
Rule of 78 (declining)	`interest − r78base` each period	`Amortize:1414`
Odd first period (prorated)	`p · (f−1) · prorate`	`Amortize:1531`

In interest-in-advance loans the payment is made at the start of the period, so interest is discounted by the factor `(f−1)/(2−f)`; `PrepaidInterest` `AMORTOP:258` handles the up-front settlement interest. The odd first period arises when the gap from settlement to the first payment is not a whole period; `prorate` `Amortize.pas:1531` is that gap measured as a fraction of a normal period, and it scales the first period's interest only.

6.5 Rule of 78

The Rule of 78 (sum-of-the-digits) front-loads interest. MakeTable computes the base unit `Amortize.pas:1414`:

r78base = (nperiods · d − amount) / (½ · nperiods · (nperiods + 1)) (the half is applied first to avoid integer overflow). The first period's interest is `r78base · (nperiods + 1)` and each subsequent period decrements by a constant `r78base` — a linearly declining interest charge whose total equals all finance charges.

6.6 Final payment & emission

Rounding leaves a sub-dollar residual at the end. Two places fold it into the last payment: `RepayFancyLoan` `AMORTOP.pas:1446` when the balance drops below `minpmt = 1.0` `(AMORTOP.pas:44)`, and `PrintAndReset` `AMORTOP.pas:1192` at `very_last` (`payamt := payamt + principal; principal := 0`, `:1207`). `DetermineVeryLast` `AMORTOP.pas:1547` computes the schedule terminus (the latest of last regular date, last balloon, every prepay stop). The irregular final amount is reported by `ReportFinalPayment` `AMORTOP.pas:1101`.

`MakeTable` `Amortize.pas:1718` emits the schedule: it calls `Enter(no_tab)` to solve, prints an optional settlement / prepaid-interest line, then either delegates to `RepayFancyLoan` (fancy / exact / non-360) or runs an inline plain loop that applies the Rule-of-78, odd-first-period, and in-advance branches above. `CloseUpShop` `Amortize.pas:982` prints the final-payment note, the subtotal, and the grand totals — and re-flags the screen as needing recalc, because the repayment walk consumed the advanced-option working state.

7 Backward solving

Backward solving is the program's reason for existing: solve for the one blank field. Most solvers share a two-step shape — a closed-form estimate, then a numerical refinement against the real schedule.

7.1 The annuity formula

The closed-form estimates are ordinary-annuity algebra. Given `nrepay` payments at periodic rate `(f−1)` on principal P (net of the present value of any balloons/prepayments), the payment solve `Amortize.pas:514–518` is:

```
denom = 1 - e^(-nrepay*ln f) = 1 - f^(-nrepay)
d = P * (f - 1) / denom     (degenerates to P/nrepay when denom ~= 0, the zero-rate case)
```

This is the textbook $d = P \cdot i / (1 - (1+i)^{-n})$ with i = f−1. The inverse, used to solve the loan amount `Amortize.pas:554`, is $amount = (1 - f^{-nrepay})/(f-1) \cdot d + padj$. The term solve inverts it for n `AMORTOP.pas:1660`: $n = round(1.4999 + \ln(1 - P(1-v)/(v\ d)) / \ln v)$ with v = 1/f (the discount factor; +1.4999 rounds up and separates the odd first period). Prepayment streams are valued with `FirstLastAndFF` `Amortize.pas:475`, which builds three discount factors so a stream's present value is $payment \cdot (first - last \cdot ff)/(1 - ff)$.

7.2 Iterate — the universal solver

When balloons, prepayments, exact day-counts, ARMs, moratoria or targets make the closed form inexact, the estimate is refined by `Iterate` `AMORTOP.pas:1690` — a secant ("Newton by difference") method whose unknown `x` is passed by reference (it can be the payment, a balloon amount, the loan amount, or the rate).

- **Objective.** Run a full silent repayment walk (`RepayFancyLoan` or `RepayLoan`) and read the terminal principal balance p. The root is p = 0 — a fully-retired loan.
- **Step.** Perturb x by $delta = small \cdot x$ ($small = 0.001$ locally), then take secant steps $newdelta = delta \cdot p / (final - p)$, tracking the best-so-far `bestx`/`bestp`.
- **Convergence.** Stop when count ≥ 20, the best terminal balance $bestp < halfpenny$ (0.005), or the rate runs away (|loanrate| > 2). The divergence brake adds 5 to `count` when the step stops shrinking.
- **State hygiene.** Each pass restores the balloon/prepay/adjustment cursors via `saved_balloon_state.Restore`, and `hard_payment` is forced false during iteration (so interest is not penny-rounded mid-search, `:1708`) and restored after (`:1771`).

> **The convergence criterion is "zero terminal balance," not a formula residual.** Because the objective is the real forward walk — advanced options and all — the solver is correct for arbitrarily complex loans at the cost of repeated schedule walks. The Go port's `fancybisect.go` reproduces this same drive-the-schedule-to-zero criterion.

7.3 The solver catalogue

Solves for	Estimate → refine	Line
Payment	Annuity inversion; closed form is exact for a plain non-exact prepaid loan (shortcut), else `Iterate(d)`.	`Amortize: 489`
Loan amount	Annuity PV of payment + extras; shortcut for plain loans, else `Iterate(amount)`.	`Amortize: 554`
Rate	First guess `payamt·peryr/amount` (floored at 0.02 — Iterate cannot start at zero), then `Iterate(loanrate)`.	`Amortize: 595`
Term	Closed-form annuity term (plain) or repay-to-payoff and read the last date (fancy).	`AMORTOP: 1589`
Balloon amount	If terminal: zero it, repay once, read the leftover off the final payment (no iteration). Else first guess ½·amount then `Iterate`.	`Amortize: 784`
Prepayment amount	Annuity inversion against the residual principal, then `Iterate`.	`Amortize: 827`
Prepayment duration	Solve the geometric series for the period count (closed form, no Iterate); requires "plus regular = No."	`Amortize: 879`
APR with points	Its own secant loop on loan value (not terminal balance): discount every payment at the trial rate; report `YieldFromRate`.	`Amortize: 659`

> **The APR solver is the exception.** Every other solver drives terminal balance to zero through Iterate. `EstimateAndRefineAPRwithPoints` instead runs its own secant loop whose objective is the present value of the payment stream matched to the net cash advanced. Don't assume it shares Iterate's machinery.

7.4 ARMs & Re_Amortize

An adjustable-rate loan applies a rate and/or payment change partway through the walk. `Re_Amortize(var p)` `AMORTOP.pas:1783` is called recursively from inside the forward walk whenever a payment date crosses an adjustment date. It resets the balance from the saved `Payment`, installs the new rate (recomputing `truerate`), and either takes the user's adjusted payment or computes a fresh annuity payment for the remaining periods — refining with `Iterate` if balloons/prepays/exact-day counting make the closed form inexact — then resumes the walk and advances to the next adjustment.

8 A worked trace

To make the flow concrete, follow a simple loan: $10,000 borrowed, 8% annual rate, 12 monthly payments, payment left blank (solve for the payment). No advanced options, so `fancy = false`.

#	What happens	Code
1	User types amount, rate, dates, pmts/yr = 12, leaves PayAmt blank. Each cell → `AssignAMZLoanValues`; the blank PayAmt becomes `payamtstatus := empty`.	`AmortizationScreenUnit.pas:1573`
2	Enter pressed → `DoCalculation` publishes `nlines` (all advanced counts 0) and calls `Enter(no_tab)`.	`AmortizationScreenUnit.pas:615`
3	FirstPass: first-date present, #periods = 12 → lastdate computed via `AddNPeriods`, `lastok := true`; `hard_payment := false`.	`Amortize.pas:271`
4	SufficientDataOnScreen passes (pmts/yr + rate + dates + amount present).	`Amortize.pas:1049`
5	GrowthPerPeriod: f = 1 + 0.08/12 = 1.0066667.	`AMORTOP.pas:1481`
6	Dispatch: payment is blank → else-branch → `EstimateAndRefinePayment`.	`Amortize.pas:1632`
7	Estimate: d = 10000·(f−1)/(1 − f^{-12}) ≈ $869.88. Plain non-exact loan, no extras → the closed form is exact → shortcut returns without iterating.	`Amortize.pas:514,518`
8	`payamtstatus := outp`; control returns to Enter, finishing steps run, APR reported.	`Amortize.pas:1632`
9	`AllAMZData2Grids` writes $869.88 back into the PayAmt cell, coloured as computed.	`AmortizationScreenUnit.pas:1505`
10	For a table, `MakeTable` → the inline plain loop applies per-period `interest = p·(f−1)`, `p := p + interest − d`, folding the residual into payment 12.	`Amortize.pas:1718`

Now change one thing: leave the rate blank instead. Step 6 routes to `EstimateAndRefineRate` `Amortize.pas:595`, which guesses `payamt·12/amount`, floors it at 0.02, and calls `Iterate` — each pass repaying the loan and nudging the rate until the terminal balance is within a half-penny of zero. Add a balloon, and even the payment solve loses its shortcut: the estimate subtracts the balloon's present value, then `Iterate` drives the real schedule (now including the balloon via ComputeNext's step 3) to zero.

9 Maintainer notes & known quirks

Behaviours that are easy to miss and have bitten ports before. None are bugs in the DOS engine unless marked; several are deliberate and load-bearing.

Penny-rounding is conditional and stateful. Interest is rounded to the cent (Round2, round-half-down) only when `hard_payment` is set — i.e. the user typed a hard payment `Amortize.pas:409, AMORTOP.pas:798`. Iterate forces `hard_payment := false` during a search and restores it after `AMORTOP.pas:1708,1771`, so the refinement runs on un-rounded interest while the printed schedule rounds. The "Dav Holle provision" `Amortize.pas:1687` additionally hardens balloon and adjusted-payment amounts to the cent only when the top-line payment is hard. Reproduce both conditions or pennies will drift.

Target overrides skip-month. When both a minimum-principal-reduction target and a skip-month apply to the same period, the target wins — a skipped month still pays target + interest `AMORTOP.pas:813`. This is intentional DOS behaviour.

Whole-dollar prepayment amounts. Prepayment payment amounts are parsed with `StringFormat2Int` (whole dollars) `AmortizationScreenUnit.pas:2078`, unlike every other dollar field, which uses `StringFormat2Double`. A port that treats them as decimals will diverge by cents.

Day-count basis is not fixed. The amortization screen runs in `iAMZ` mode, whose default is the actual/actual 365/366 loan rule (§5.3, branch C), not 30/360. The basis is user-selectable and changes both the interest interval and the in-advance shape. There is no single "days in a year" constant to hard-code.

Fancy mode: explicit toggle (DOS) vs auto (port). In the DOS source the user must toggle Advanced on to engage fancy; mere presence of an advanced value is not enough `AmortizationScreenUnit.pas:487`. The Go port sets `Fancy = true` whenever any advanced option is supplied. The numeric results agree; the routing differs.

Odd-first-period: DOS augments, Windows help does not. For a long/short first period the DOS engine prorates and augments the first payment's interest (`prorate`, §6.4). The Windows help text describes a different handling. The Go port follows DOS. This is documented in the project's discrepancies log.

Latent quirks (left as-is in the source). `YearsDif` exists in three historical versions in the file `INTSUTIL.pas:946,996`; only the last ("adopted 12/93", line 996) is live — don't cite the wrong one. The date epoch is 1900 for the Julian/MDY math, but the modernized `SetNow` uses a −1950 offset `VIDEODAT.pas:408` — an artifact that does not enter the serial-date formulas.

10 Appendix: call graph & file:line index

```
Amortize.Enter(code)                                    Amortize.pas:1104
  |- FirstPass                                          Amortize.pas:271
  |    +- CheckPrepayments / SortAdj / SortBalloons / MonthSetFromString
  |- SufficientDataOnScreen                             Amortize.pas:1049
  |- GrowthPerPeriod / DetermineVeryLast                AMORTOP.pas:1481 / 1547
  |
  |- DISPATCH (solve the one blank field):              Amortize.pas:1577
  |    |- EstimateAndRefinePayment                      Amortize.pas:489 --+
  |    |- EstimateAndRefineRate                         Amortize.pas:595 --|
  |    |- EstimateAndRefineLoanAmount                   Amortize.pas:554 --|
  |    |- EstimateAndRefineBalloon                      Amortize.pas:784 --+-> Iterate (secant)
  |    |- EstimateAndRefinePeriodicPrepayment           Amortize.pas:827 --|     AMORTOP.pas:1690
  |    |- DeterminePrepaymentDuration (closed form)     Amortize.pas:879 --|       +-> RepayFancyLoan / RepayLo
  |    |- DetermineLastPaymentDate                      AMORTOP.pas:1589 -+             AMORTOP.pas:1331 / 152(
  |    +- EstimateAndRefineAPRwithPoints (own secant)   Amortize.pas:659              +-> ComputeNext
  |                                                                                        AMORTOP.pas:723
  +- finishing: per-adj solves / APR / balance-as-of / Dav Holle

MakeTable                                               Amortize.pas:1718
  |- Enter(no_tab)
  |- RepayFancyLoan(entire, Output) -> ComputeNext
  |                                      +- (at adj date) Re_Amortize AMORTOP.pas:1783
  +- CloseUpShop -> ReportFinalPayment / PrintGrandTotals    Amortize.pas:982
```

Symbol	Reference	Section
Status codes empty/outp/defp/inp	PETYPES.PAS:178,216–218	1.2, 2.1
Global model containers (h, pre, ...)	PEDATA.pas:123,233,257–277	2.2
paymenttype object	AMORTOP.pas:87	2.3
AssignAMZLoanValues / DoCalculation	AmortizationScreenUnit.pas:1573 / 615	3.2, 3.4
FirstPass / SufficientDataOnScreen / Enter	Amortize.pas:271 / 1049 / 1104	4
Dispatch cascade	Amortize.pas:1577–1675	4.4
Round2	INTSUTIL.pas:346	5.1
Julian / MDY	VIDEODAT.pas:470 / 490	5.2
YearsDif (live version)	INTSUTIL.pas:996	5.3
AddPeriod / NumberOfInstallments	INTSUTIL.pas:1543 / 1210	5.4
exxp / lnn / Power	INTSUTIL.pas:1460 / 1487 / 331	5.5
YieldFromRate / RateFromYield	INTSUTIL.pas:1612 / 1624	5.6
GrowthPerPeriod	AMORTOP.pas:1481	6.1
ComputeNext	AMORTOP.pas:723	6.2
RepayFancyLoan / RepayLoan	AMORTOP.pas:1331 / 1520	6.3
MakeTable / CloseUpShop	Amortize.pas:1718 / 982	6.6
Iterate (secant solver)	AMORTOP.pas:1690	7.2
Re_Amortize (ARM)	AMORTOP.pas:1783	7.4

Generated for the Per%Sense DOS→Go port. Line numbers cite `legacy/src_documented/dos_source/`. For the present-value and mortgage engines, see their respective units (`PRESVALU.pas`, `Mortgage.pas`), which follow the same field-presence-dispatch pattern documented here.